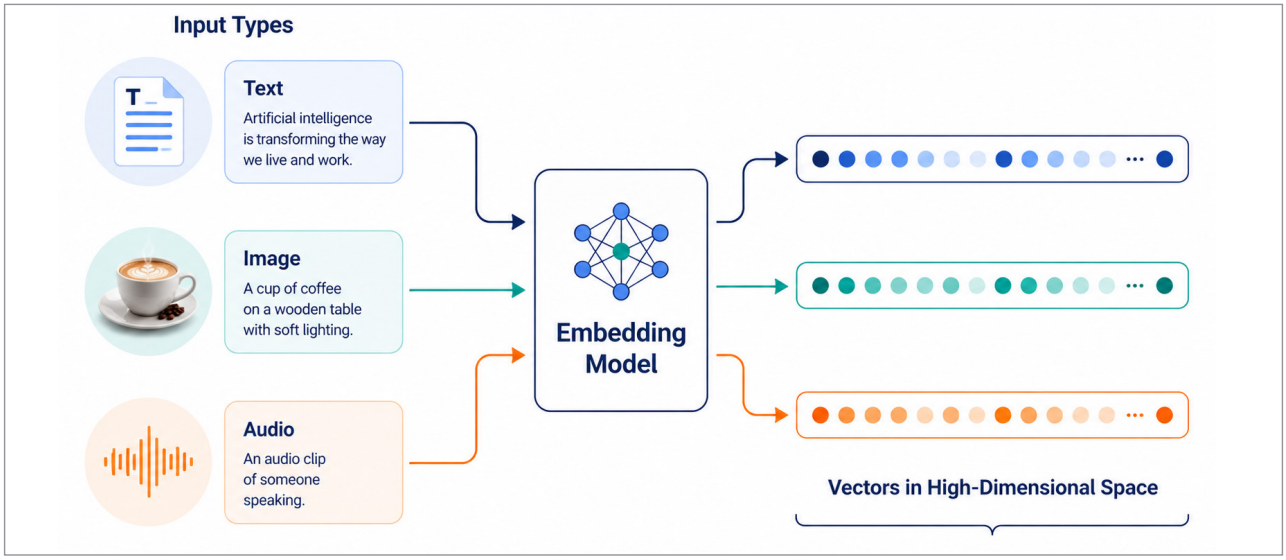

Figure 1: How embeddings turn text and image into searchable vectors

questions. They're similarity questions. And SQL is not great at similarity, at least not the kind of meaning-based similarity AI introduced.

To handle these questions, you first turn whatever you're searching -- text, images, audio, anything really — into a long list of numbers called a vector or an embedding. Different inputs produce different vectors. Things that mean similar stuff produce vectors that sit close to each other in this high-dimensional space.

The vector database's job is simple but tricky. Store millions or billions of these vectors. When somebody hands you a new vector, find the ones that are closest to it. Quickly. That's basically it.

It sounds straightforward. It is not. Doing this fast at scale requires some genuinely clever data structures and algorithms, which is why vector databases became their own thing rather than just a feature of existing databases.

Why this got popular almost overnight

Vector search isn't new. Recommender systems have been doing variations of this for fifteen years. What changed recently is the availability of good embedding models.

A few years ago, getting a decent vector representation of a piece of text required training your own model, which was expensive and finicky. Now you can call an API or run an open source model locally and get state-of-the-art embeddings for basically nothing. Sentence-transformers, OpenAI's embedding models, Cohere, BGE, Instructor, GTE -- there are dozens of solid options.

Once embeddings became cheap and easy, the bottleneck moved to storing and searching them. Hence the explosion of vector databases.

The other accelerant has been the rise of retrieval-augmented generation. RAG, if you want the acronym. The basic idea is, instead of trying to fine-tune a giant language model on your specific data, you keep the model generic and store your data as embeddings. When a user asks a question, you find the relevant chunks of your data via vector search and stuff them into the prompt. The model then answers using that context.

This pattern turns out to be enormously useful for chatbots, internal documentation search, customer support tools, and a hundred other things. And every one of those needs a vector database underneath it.

What's inside a vector?

Let me get a bit more concrete about what an embedding looks like. You take a sentence. Say, "The cat sat on the mat." You feed it into an embedding model. What comes out is a list of numbers, usually somewhere between 256 and 1536, depending on the model. Each number is just a floating-point value, something like 0.0234 or -0.4891. Looking at the list, you can't make any sense of it. The numbers don't correspond to anything you can name.

But here's the magical bit. If you feed in "A feline rested on the rug," you get a different list of numbers. And if you computed the mathematical distance between these two lists, treating them like points in a 768-dimensional space, that distance would be small. Much smaller than the distance between "The cat sat on the mat" and "The stock market opened higher today."

The embedding model learned, through training on huge amounts of text, that semantic meaning can be encoded as positions in this weird high-dimensional space. Things that mean similar stuff end up near each other. Things that mean different stuff end up far apart.

Vector databases store these lists, billions of them sometimes, and let you